

RxInsight - Interview Questions

Questions and model answers on this project and its fundamentals

RxInsight · github.com/Swapnil-byte-798/rxinsight · Swapnil Dwivedi

Preparation notes for defending this project in a Business Technology Solutions Associate interview. Every number here was measured on the machine that built the project, or queried from the loaded database while writing this. Where something did not work, the answer says so.

1. The project in brief

Q: Walk me through this project.

A: RxInsight is a pharmaceutical commercial analytics warehouse: a synthetic vendor-style feed, a Python ETL, a PostgreSQL star schema, and four commercial questions answered with window-function SQL. It holds 2,229,747 prescription rows and 68,619 sales-call rows against four conformed dimensions — date, product, territory and HCP — with the HCP dimension as Slowly Changing Dimension Type 2, so a doctor who moves territory in June does not retroactively rewrite January's numbers. What I would actually defend is the measurement discipline: I benchmarked the indexes properly, and one of the two results was a 42x win while the other was flat — I published both.

Q: Give me the sixty-second version.

A: Two million rows loaded into a Postgres star schema by a pipeline that quarantines bad rows instead of dying on them — 2,230,247 staged, 2,229,747 loaded, 500 rejected, asserted every run. The HCP dimension is Type 2 and key resolution is temporal, so each prescription points at the version of the doctor current the day it was written. Loading with foreign keys enabled never finished — I killed it at 14 minutes 48 seconds — while suspending and revalidating them does it in 41 to 66 seconds. On top sit four queries, thirty-six tests, and a benchmark that honestly reports the flagship index doing nothing for the aggregate it was built for.

Q: Why did you build it?

A: I wanted to learn what a commercial analytics stack in life sciences looks like, and the only way to learn that is to build one and hit the problems. Pharma forces interesting modelling decisions: territory alignments change, deciles get resegmented, and a warehouse that overwrites history gets the sales numbers wrong. I also wanted claims I could measure rather than assert.

Q: What was the hardest part?

A: Measuring honestly. My first benchmark showed a 1.9x speedup on the brand-share aggregate and I was ready to publish it — it was cold cache versus warm cache, because I ran the un-indexed case first. When I rewrote the harness to discard a warm-up run and take the median of three, the speedup went to zero.

Follow-up they may ask: Why not just delete the negative result? A benchmark I cannot reproduce in front of you is worth less than no benchmark.

2. Data modelling

Q: Star schema versus snowflake — what is the difference, and which did you use?

A: A star has fact tables carrying measures surrounded by flat dimension tables; a snowflake normalises those dimensions into sub-tables, so region would leave `dim_territory` and become its own table. I kept mine flat because the dimensions are tiny — 50 territories, 8 products — so the storage saving is meaningless and every extra join is pure cost. I would snowflake a genuinely large dimension with a big repeating sub-structure.

Q: How do you decide what is a fact and what is a dimension?

A: A fact is something you measure and aggregate; a dimension is something you filter or group by. My test is whether summing it means anything — summing `trx_count` gives total prescriptions, summing `decile` gives nonsense. Facts are numeric, high-volume and append-only; dimensions are textual, small and slowly changing.

Q: What is the grain of your fact tables, and why does grain matter?

A: `fact_prescriptions` is one row per HCP per product per day; `fact_sales_calls` is one row per call. Grain is the first decision and the one you cannot undo cheaply, because it fixes what you can ever ask. Had I pre-aggregated to the month, the 14-day post-call lookback in `call_effectiveness.sql` would be impossible.

Q: What is a conformed dimension?

A: A dimension shared by more than one fact table with identical keys and meaning. All four of mine are conformed across prescriptions and sales calls, which is what makes the call-effectiveness query possible — I can join calls to prescriptions on `hcp_key` because both facts mean the same thing by that key. Without conformity you get two reports that both say "territory" and quietly disagree.

Q: Surrogate key versus natural key — why bother with surrogates?

A: A natural key comes from the source system, like `HCP-00742`; a surrogate is an integer the warehouse assigns and controls. Surrogates are narrow and stable, so a source renumbering does not destroy my history. Critically, Type 2 is impossible without them: one `hcp_id` has several versions and each needs its own key for facts to point at.

Q: Why is your date key an integer, `20240131`, instead of a DATE?

A: It is compact, it sorts and ranges correctly, and it still reads as a date to a human debugging a fact row. It also keeps all calendar logic — month names, quarters, the `year_month` grouping string — in `dim_date` rather than recomputed by every query. The risk is that it tempts people into arithmetic like `date_key - 14`, which is wrong across a month boundary, so my lookback joins through `dim_date` to real dates first.

Q: Warehouses lean denormalised. Why, when normalisation is what we are taught?

A: Normalisation optimises for write integrity, which is right for an OLTP system taking thousands of small writes a second. A warehouse is written once in a batch and read constantly with large aggregating scans, so the update anomaly barely exists while every extra join costs query time. I take that trade once more: `territory_key` sits on the fact even though `dim_hcp` could supply it, because storing the territory that owned the script that day makes the historically correct answer the easy one to write.

Q: Additive, semi-additive, non-additive — explain with your own columns.

A: Additive means you can sum across every dimension: `trx_count`, `nrx_count`, `units` and `gross_sales` all qualify. Semi-additive sums across some dimensions but not time — a stock balance, summed across branches but taken period-end over time. Non-additive cannot be summed at all: `share_pct` and `attainment_pct` are ratios, so they must be recomputed from summed numerator and denominator at each level.

3. Slowly Changing Dimensions

Q: What are Type 1, Type 2 and Type 3?

A: Type 1 overwrites in place, so history is lost and every historical report silently changes to match today's value. Type 2 closes the existing row and inserts a new version with its own surrogate key and validity dates, preserving history exactly. Type 3 keeps a "previous value" column beside the current one, giving exactly one step of history.

Q: Why is Type 2 right here, and what breaks without it?

A: Territory ownership is what reps are paid on. If Dr. Sharma moves from T-12 to T-19 in June and I overwrite her row, every prescription she wrote in January retroactively belongs to T-19, and the rep who covered T-12 loses credit for work they did. The failure is quiet, which makes it dangerous — nothing errors, the numbers are simply

wrong and they change on every reload.

Q: How does your implementation work?

A: `apply_scd2` in `etl/transform.py` takes the current dimension state and the incoming rows and does one of three things per row: no current version, insert as current; a tracked attribute changed, close the live row with `valid_to` set to the day before the effective date and append a new open row; nothing tracked changed, do nothing. On this load, 2,000 distinct HCPs produced 2,038 rows with 38 closed history rows.

Follow-up they may ask: Why track only decile and territory? Versioning on everything means a typo correction forks history for no analytical benefit.

Q: Explain the partial unique index and why you used one.

A: A plain unique index on `hcp_id` would be wrong, because historical rows deliberately repeat it. The `WHERE` clause narrows the constraint to the current slice, so "exactly one live version per doctor" is enforced by the database rather than trusted from my Python, and the index stays small — one entry per HCP, not one per version.

```
CREATE UNIQUE INDEX uq_hcp_current ON warehouse.dim_hcp (hcp_id) WHERE is_current;
```

Q: What is the temporal join, and how do you know it actually happened?

A: Resolving a prescription's `hcp_id`, I take the version whose validity window contains the prescription date, not the current one, and the territory comes from that same version. I prove it with an assertion over all 2.2 million rows: join every fact to its HCP version and to `dim_date`, count rows where the fact date falls outside the window, and it returns zero. A naive "latest version wins" load would fail that on every fact belonging to a closed version — 16,689 rows here, 0.75% of the table, which is small enough to pass unnoticed and large enough to move a territory's numbers.

Q: What is a late-arriving fact, and how does yours behave?

A: A fact that arrives after the dimension moved on — a March prescription landing in July, after a May territory change. Because my lookup is by date rather than current version, it still resolves to the March version correctly. What I do not have is a late-arriving *dimension* path: if the master reports a retroactive change, facts already loaded are not rebuilt.

4. SQL and RDBMS fundamentals

Q: Window function versus GROUP BY — when do you need a window?

A: `GROUP BY` collapses rows; a window keeps every row and adds a value computed over a related set. The moment you need the detail line and the aggregate on the same row, you need a window. In `brand_share_mom.sql` I need each brand's monthly TRx beside the whole market's total, and `SUM(trx) OVER (PARTITION BY year_month)` gives that denominator without a self-join.

Q: What does NTILE do, and where do you use it?

A: `NTILE(n)` sorts the partition, splits it into `n` as-equal-as-possible buckets and labels each row with its bucket. I use `NTILE(10) OVER (PARTITION BY territory_key ORDER BY total_trx DESC)` for each prescriber's decile within their own territory. It has to be within territory, because a rep can only call on doctors in their own geography.

Q: Explain LAG and how you used it.

A: `LAG` reads a value from an earlier row in the same partition, following the window's ordering. `LAG(share_pct) OVER (PARTITION BY brand_name ORDER BY year_month)` gives last month's share for the same brand, and subtracting yields the month-over-month delta in percentage points. Partitioning by brand stops brand A's January

being compared with brand B's December, and the first month correctly yields NULL rather than a fake zero.

Q: RANK versus DENSE_RANK versus ROW_NUMBER.

A: All three number rows within a partition and differ only on ties. ROW_NUMBER never ties, so two equal values get 1 and 2 arbitrarily; RANK gives ties the same number then skips, so two firsts are followed by third; DENSE_RANK gives ties the same number and does not skip. I use RANK for the territory league table because identical attainment genuinely shares a position.

Q: CTE versus subquery — is a CTE faster?

A: No, and that is the honest answer. In modern Postgres a non-recursive CTE is inlined by default, so it is usually the same plan as the equivalent subquery and the real difference is readability — my call-effectiveness query is six stacked CTEs and would be unreviewable as nested subqueries. MATERIALIZED forces the old behaviour, which matters when a CTE is referenced several times.

Q: Walk me through the join types.

A: INNER keeps only matches; LEFT keeps every left row with NULLs where nothing matched; RIGHT mirrors it; FULL OUTER keeps unmatched rows from both; CROSS is the Cartesian product. The one carrying analytical meaning here is the LEFT join in call_effectiveness.sql — prescribing days with no preceding call must survive as the control cohort, and an inner join would silently delete the comparison group and leave me reporting a lift against nothing.

Q: What does ACID mean, and how do transactions and isolation fit in?

A: Atomicity is all or nothing, consistency means moving between valid states with constraints satisfied, isolation means concurrent transactions do not see each other's uncommitted work, durability means a commit survives a crash. Isolation is a dial — Read Uncommitted, Read Committed, Repeatable Read, Serializable — defined by the anomalies each prevents, with Postgres defaulting to Read Committed. My pipeline leans hardest on atomicity: the fact COPY and the constraint revalidation are one transaction, so a failed revalidation rolls the facts back and no unvalidated row is ever visible. Staging and the dimensions are committed before that, so it is not a single transaction end to end — the next run drops and recreates both schemas anyway.

Q: Tell me about index types — B-tree, partial, covering.

A: A B-tree is a balanced tree whose leaves are linked in order, so one structure serves equality lookups, range scans and ORDER BY. A partial index carries a WHERE clause and indexes only matching rows, like uq_hcp_current. A covering index carries extra non-key columns via INCLUDE so a query can be answered index-only without touching the heap — idx_rx_date_product includes trx_count for that reason, and, said plainly, it did not pay off for the query I built it for.

Q: Primary key, unique constraint, foreign key — what is actually different?

A: A primary key is the one identifying column set: unique, not null, one per table. A unique constraint also enforces uniqueness but you can have several, and in Postgres it permits NULLs, which are not equal to each other, so a nullable unique column can hold many NULLs. A foreign key is referential integrity across tables — my facts have a surrogate primary key and four foreign keys each, which is where the load-performance story comes from.

Q: Explain NULL semantics and one place they bite.

A: NULL means unknown, and any comparison with it yields unknown rather than true or false, so = NULL never matches and you need IS NULL. It propagates through arithmetic, and aggregates skip it, so AVG divides by the non-null count. Where it bites me is division: every ratio in the four analytics queries wraps its denominator in NULLIF(x, 0) — territory_attainment.sql, call_effectiveness.sql and brand_share_mom.sql — so a zero target produces a blank cell instead of killing the report.

5. Performance

Q: How do you read an EXPLAIN ANALYZE plan?

A: EXPLAIN gives the planner's estimate, EXPLAIN ANALYZE runs it and gives real times and row counts. I read it inside out, because the innermost nodes execute first, and I look at the node types, whether estimated rows match actual rows, and where the time goes. A big estimate-versus-actual gap means stale statistics, and BUFFERS shows how much came from cache versus disk.

Q: When is a sequential scan correct, and when do you want an index scan?

A: A sequential scan reads pages in order, which is fast per page and parallelisable; an index scan is random access, and each hit costs a descent plus a heap fetch. Past roughly a few percent of the table random access loses, so a large aggregate wants the seq scan and a narrow lookup wants the index. My brand-share query aggregates every row in a 2.2-million-row table, so a parallel sequential scan there is the right answer, not a failure to optimise.

Q: Which of your indexes paid off, and which one did nothing?

A: etl/measure.py times two queries with all four indexes dropped and then recreated. The selective one paid off: WHERE hcp_key = 742 went from 146.9 ms to 3.5 ms with idx_rx_hcp — 42 times faster — and the plan changed from a parallel sequential scan to a bitmap index scan feeding a bitmap heap scan. The monthly aggregation at the core of brand_share_mom.sql — the scan-and-group-by that dominates that report — went from 2181.1 ms to 2139.6 ms, which is noise, and stayed a parallel sequential scan both ways. The difference is selectivity: the drill-down touches 1,422 rows out of 2,229,747, or 0.064%, while the aggregate touches all of them, and an index only helps by letting you skip rows.

The four numbers worth having by heart:

Query	No index	Indexed	Change	Plan when indexed
Prescriber drill-down (hcp_key = 742)	146.9 ms	3.5 ms	42x faster	Bitmap Heap Scan
Monthly aggregation behind brand_share_mom.sql	2181.1 ms	2139.6 ms	no change	Parallel Seq Scan

Follow-up they may ask: Then what would help the aggregate? Partition pruning or a pre-aggregated monthly rollup — not an index.

Q: How do you order the columns of a composite index?

A: Equality predicates first, range predicates last, because a B-tree can only range-scan a prefix of its columns. idx_calls_hcp_date is (hcp_key, date_key) for that reason: pin one HCP with equality, then scan a date window inside that slice. Reversed, the date range would lead and hcp_key would degrade into a filter applied to every row in the period.

Q: Tell me about the foreign-key bulk-load finding.

A: My first fact load with all eight foreign keys enabled never completed — I killed it at 14 minutes 48 seconds. Four foreign keys on 2.2 million rows means roughly 8.9 million per-row constraint checks, each an index lookup into a dimension. Dropping the constraints around the COPY and re-adding them afterwards replaces that with one bulk validation pass per constraint, and the load finishes in 41 to 66 seconds. The guarantee is unchanged: the ADD CONSTRAINT runs inside the same transaction, so one violating row rolls the whole load back.

Follow-up they may ask: Would you do that in production? Only for a controlled full load into a table nobody is reading; for incremental loads the constraints stay on.

Q: What would you do at 200 million rows?

A: Partition `fact_prescriptions` by month on `date_key`. A query bounded to a quarter then touches three partitions instead of the whole table — genuine pruning, rather than an index trying to narrow something unnarrowable, which is exactly where my composite index did nothing. It also makes retention cheap, since dropping an old month becomes a `DROP TABLE` rather than a huge `DELETE` and vacuum.

6. Python and ETL

Q: Why is all your transform logic in pure functions?

A: Every function in `etl/transform.py` takes DataFrames and returns DataFrames — none open a connection, read a file or look at the clock. That means the SCD2 logic can be tested against five hand-written rows in milliseconds with no Postgres near the test, which is why 15 of my 36 tests need no database at all.

Q: Why `COPY` instead of `pandas.to_sql`?

A: `to_sql` issues parameterised INSERTs. Pandas batches them through `executemany`, so it is not literally two million round-trips, but it is still two million rows of statement parsing and parameter binding. `COPY FROM STDIN` streams the whole thing as one buffered operation the server parses in bulk, and on a warehouse load that difference is the entire job. I stream it in chunks of 250,000 rows, because serialising two million rows into one in-memory buffer costs hundreds of megabytes for no gain.

Q: How do you handle bad rows?

A: Staging is deliberately all `TEXT` with no constraints, so a malformed value cannot kill the `COPY` at the boundary. Casting happens in Python, where `coerce_columns` returns clean rows plus rejects with a reason attached, and key resolution does the same for unresolvable products or HCPs. On this load that caught 500 rows — 250 with `N/A` in `trx_count`, 250 with reason unknown `product_code` — written to `data/rejects/` rather than dropped. The counts always balanced; the `contents` of that second file did not until I fixed the bug in section 9.

Follow-up they may ask: Why not fail the whole load on a bad row? One bad row out of 2.2 million should cost you that row, not the night's load, as long as it is counted and visible.

Q: Is your pipeline idempotent?

A: Yes, but bluntly so: it drops and recreates both schemas at the start, so running it twice gives the same result rather than doubled facts. That is honest full-refresh idempotency, not sophistication. The generator is seeded at 42, so the source data is byte-identical between runs, which is what makes the measurements comparable at all.

Q: How would you make the load incremental?

A: Keep a watermark — the highest `date_key` successfully loaded — pull only rows above it, and record each run in a load-audit table. Facts append, so that is easy; the dimension needs an upsert running the same SCD2 comparison against existing state, which `apply_scd2` already supports since it takes existing state as its first argument. I would also handle restatements explicitly, because pharma feeds get restated.

Q: What is the memory behaviour of your pipeline?

A: It is the honest weak point — the full prescription frame is in memory at once, fine at 2.2 million rows on a laptop but not linearly scalable. The chunking in `copy_dataframe` bounds the serialisation buffer, not the frame itself. The real fix is date-bounded chunks end to end, or pushing the transform into SQL against staging.

7. Testing and data quality

Q: What do you test, and why those things?

A: Thirty-six tests in two groups. Fifteen are pure unit tests over the transform functions: that a tracked change creates a version and an untracked rename does not, that a bad cast is quarantined rather than raised, that loaded plus rejected always equals input, and — added after the bug in section 9 — that every rejected row

actually exhibits the reason it was labelled with, even when the frame's index has gaps. Twenty-one are data-quality assertions against the loaded warehouse — reconciliation, orphan foreign keys across eight relationships, null and non-negativity thresholds, `nrx_count <= trx_count`, and the SCD2 invariants.

Q: What is the reconciliation identity and why does it matter?

A: Staged equals loaded plus rejected — 2,230,247 equals 2,229,747 plus 500. It is the difference between a pipeline and a hope: without it, rows disappear in a join or a filter and nothing tells you, totals come out slightly low, and nobody notices for a quarter. The pipeline prints all three numbers every run and raises if they do not balance.

Q: You have foreign keys. Why also test for orphan keys?

A: Because the constraints are dropped during the bulk load, so the test verifies the revalidation really did its job rather than assuming it. It also protects against someone later loading with constraints off and forgetting to put them back.

Q: How would you run these in production, and what is a data-quality SLA?

A: As a gate between load and publish, not a report after the fact: the load writes, the checks run, and only if they pass does the data become visible — otherwise it rolls back and alerts. An SLA is the agreement with consumers about what "good" means in numbers rather than adjectives: freshness, like landing by 6am; completeness, like row counts within a tolerance of source; and validity thresholds. My tests are the assertion half; the SLA is the agreement about which may fail and what happens then.

8. Domain and business

Q: Define HCP, TRx and NRx.

A: HCP is a health care professional — commercially, the prescriber the field force calls on. TRx is total prescriptions in a period, new plus refills, and it is the headline volume measure. NRx is new prescriptions only, and it is the leading indicator, because a brand can hold TRx steady on its refill base while its NRx collapses.

Q: What is a decile, what is a territory, and why do they matter commercially?

A: A decile segments prescribers into ten groups by volume; a territory is a geographic book of business assigned to one rep, and mine has 50 across 2,000 HCPs. They matter because a rep's time is the scarcest resource in the commercial model — you cannot call on everyone, so you rank prescribers and spend face-to-face time where the volume is while the tail gets e-detailing. The territory is the unit of accountability for quota and incentive compensation, which is why reassignment has to be Type 2 history rather than an overwrite.

Follow-up they may ask: Which end of the decile is the top? In my `dim_hcp.decile` column 10 is the highest — I checked, and decile 10 averages 2,568 own-brand TRx against 45 for decile 1.

Q: What would a brand team actually do with each of your four reports?

A: The decile ranking is the call plan — it decides who gets a visit. Brand share is the number the brand is measured on, because absolute TRx can rise while share falls in a growing market. Territory attainment is the field-force scorecard for the monthly business review, where the YTD column matters more than the monthly one because one month is noise. Call effectiveness is the return-on-promotion question that decides next year's field-force budget.

Q: Your call-effectiveness result — what does it say, and how much do you believe it?

A: Called HCP-months average 39.31 TRx against 23.71 for uncalled, a 65.8% lift, and the lift is largest in the low deciles: 94.5% in decile 1 down to 30.7% in decile 10, non-increasing the whole way, though deciles 7 and 8 tie at 34.9%. That direction is the commercially interesting read, not the headline — the high-volume prescribers were going to write anyway, so the marginal call is worth more at the bottom of the book than the top. As a measurement of this pipeline I believe it completely; as medicine, not at all, because the generator deliberately

plants a 14-day post-call lift and recovering it is a correctness check that the ETL preserved a known signal.

Q: Suppose the data were real. What would be wrong with that analysis?

A: It is observational and confounded in the obvious direction: reps deliberately call on high prescribers, so the called cohort is selected for people who would have written more anyway. The national lift is an upper bound, and stratifying by decile is the cheap partial control, which is why the per-decile rows are what I would defend in a meeting. There is also a survivorship quirk: an HCP-month only exists if that doctor wrote at least one script, so months of silence are invisible.

9. Hard and uncomfortable questions

Q: Is this real data?

A: No. It is entirely synthetic, generated by `etl/generate_data.py` with a fixed seed of 42, and there is no real patient, prescriber or commercial data anywhere in the repository — which is also why I could make it public. That is stated in the README's opening section and in the introduction to `docs/how-it-works.md`, not left to a line in the licence note, because in this industry a candidate who is vague about data provenance is a liability.

Q: Did you use AI to build this?

A: Yes, as a tool, the way I would use documentation or a senior colleague reviewing an approach. What I did not outsource is the judgement: making `dim_hcp` Type 2, measuring the indexes instead of asserting they helped, and publishing the result that did not flatter me. The test of whether that matters is whether I can defend the code — so ask me anything in here, including the two bugs I found in my own repository while preparing for this conversation.

Q: Your README says the index did not help. Why put that in?

A: Because it is the most useful thing I learned. Anybody can add an index and claim a win; the skill is knowing when it will not work and why, and "no selective predicate means nothing to skip" is a rule I can now apply to a query I have never seen. Leaving it out would also mean the README carried a claim I could not reproduce in front of you.

Q: You measured a 1.9x speedup that turned out to be cache. How did you catch it?

A: The plan did not change. Before and after, the node was a parallel sequential scan doing identical work, so a 1.9x difference had to come from somewhere other than the query, and `BUFFERS` confirmed it — the un-indexed run read from disk, the indexed run from a warm cache, because I had run them in that order. The fix was in the harness, not the database: discard a warm-up run, take the median of three, and the effect went to zero.

Follow-up they may ask: What is the general lesson? If the plan is identical, the time difference is environmental, not algorithmic.

Q: What is weakest about this project?

A: A real bug I found by checking rather than by being told. In `resolve_surrogate_keys` I tracked row positions but indexed the original frame with `.iloc`, which selects by label — and by the time that function runs, `coerce_columns` has already dropped the bad rows, so the index has gaps and label and position diverge. The wrong rows landed in the reject file. The counts still balanced, which is exactly why the reconciliation test never caught it, but only 4 of the 250 rows written with reason `unknown product_code` carried the bad code. It is `.iloc` now, and I found it by reading the reject file rather than by a test — which is the uncomfortable part.

Follow-up they may ask: What does that say about your tests? They checked counts, not content. The assertion that catches it — feed `resolve_surrogate_keys` a gapped index and require every rejected row to exhibit its stated reason — is `test_rejects_survive_a_gapped_index`, which I wrote with the fix and not before it.

Q: What would you do differently?

A: The second bug I found is still open: `dim_hcp.decile` runs 10 as the best prescriber, but the `calculated_decile` from `NTILE(10) ORDER BY total_trx DESC` makes 1 the best, so the two scales run in opposite directions and the comment suggesting you can compare them directly is wrong as written. Architecturally, I would build the incremental path from the start instead of full-refresh, which let me dodge the hard questions about restatements and retroactive dimension changes. And I would have written the benchmark harness before the first benchmark rather than after being embarrassed by one.

Q: How do I know you understand this rather than copied it?

A: Ask me to change something. Ask why `idx_calls_hcp_date` is ordered `(hcp_key, date_key)` and what happens reversed, what breaks if I drop the `LEFT` from the call-effectiveness join, or why the reconciliation test passed while the reject file was wrong. The two bugs are the strongest evidence I have, because I found them by probing my own repository — that is not something you get from copying, and not something you volunteer if you have not understood the code.

Questions to ask them

- 1 What does the first six months look like for a BTSA — closer to data engineering, closer to analytics, or moving between them depending on the engagement?
- 1 What does a typical engagement team look like, and where does someone at my level sit in it?
- 2 How much of the work is building new pipelines versus extending what a client already has?
- 3 What is the stack in practice — Snowflake or Redshift, dbt and Airflow, how much is still hand-written SQL?
- 1 When the data contradicts what the business believes, how does that conversation usually go, and who has it?
- 1 What separates someone doing well after a year from someone who is just keeping up?
- 2 How does ZS handle the regulatory side — patient privacy, aggregation rules on prescriber data — and how early does someone at my level encounter it?
- 1 What would you want me to have learned or read before starting, if I get the role?